

软件综合开发实训实验报告

1. 实验概述

课程名称：软件综合开发实训

实验名称：锄大地扑克牌游戏

项目名称：锄大地 Android 棋牌游戏

面向班级：2026 软件工程

开发周期：5 周开发迭代。项目阶段按照《软件开发综合实训提交时间节点 V0.0.1.xlsx》划分为开始、三次细化/构造迭代和交付。

小组成员及主要职责：

成员	主要职责
Annakuliyev Sedar	ProcessOn UML 大图绘制、图形布局调整、建模材料整理
莫均杰	Android 主体代码编写、规则模块、AI 模块、蓝牙模块、UI 页面、发布集成
郭铭涛	单元测试、蓝牙模拟测试、运行验收、测试报告、录屏验证

1.1 实验目的

本次实训围绕 Android 锄大地游戏完成软件开发全过程训练，主要目标包括：

- 理解软件开发过程，结合 UP 迭代开发方法完成需求分析、面向对象分析、面向对象设计和实现。
- 掌握 UML 建模方法，使用 ProcessOn UML 完成用例图、类图、顺序图、状态图、活动图、组件图和部署图。
- 熟悉 Android Studio、Gradle、Kotlin 和 Jetpack Compose 的开发流程。
- 实现完整的棋牌游戏功能，包括人机对局、多人蓝牙联机、南北玩法规则、AI 出牌策略、玩家成长、动画音效和发布包。
- 在开发过程中实践 SCRUM 敏捷协作，并如实记录 Codex/GPT-5 在方案设计、代码实现、UML 图绘制、Bug 修复和测试扩展中的主要参与情况。

1.2 实验环境

类型	环境说明
硬件环境	Android 4.4 以上系统手机或 Android 模拟器，牙联机收需要多台手机
开发工具	Android Studio、Gradle、Git、GitHub
建模工具	ProcessOn UML、PlantUML
编程语言与框架	Kotlin、Jetpack Compose、Android SDK
测试工具	JUnit、Gradle Test、Android Instrumented Test、蓝牙模拟测试
AI 工具	OpenAI Codex / GPT-5，用于需求拆解、主要代码实现、UML 草图/更新、测试补全和文档同步

1.3 项目简介

本项目是一款 Kotlin + Jetpack Compose 实现的 Android 锄大地棋牌游戏。系统支持本地人机对局、好友蓝牙多人房间、客户端断线人机托管、房主断线弹窗退出、南北规则切换、三档 AI 策略、玩家成长系统、开屏和牌桌动画、背景音乐、出牌音效、Release 签名包和完整 UML/文档交付。

项目采用 MVC 思想分层：Model 层负责扑克牌、牌型、规则、状态和玩家档案；Controller 层负责回合推进、AI 决策、联机同步和成长结算；View 层使用 Jetpack Compose 实现开屏、大厅、规则、设置、教程、蓝牙房间、牌桌、结算和玩家中心等页面。

1.4 当前版本完整功能基线

- 1. 开屏页：使用修补后的 CC0 扑克牌素材作为背景，玩家必须点击 开始游戏 才进入大厅。
- 2. 应用图标：使用 FreeIcons 的 Cards CC0 成品素材作为主体，并做 Android 桌面图标适配。
- 3. 游戏大厅：显示玩家头像、昵称、等级、金币、战绩入口，以及人机对局、好友蓝牙对局、规则、设置、教程等入口。
- 4. 人机对局：支持简单、普通、困难三种 AI 难度；开局前先入桌，点击 开始游戏 后播放发牌动画。
- 5. 规则系统：支持北方规则与南方规则，保存后对后续人机和蓝牙对局生效。
- 6. 蓝牙房间：支持创建房间、加入对局、四座位房间、人机补位、房主权威同步、结算后回原房间再开局。
- 7. 蓝牙稳定性：支持客户端断线 AI 托管；房主断线时客户端弹窗提示并退出当前房间，需要重新创建或重新加入。当前版本没有实现自动断线重连。
- 8. 玩家成长：支持金币、经验、等级、称号、战绩、成就、每日奖励、头像商店、自定义头像、改名消费、战绩重置。
- 9. 动画与音效：包含开屏光效、大厅漂浮扑克牌、主视觉光效、按钮按压、牌桌发牌、手牌选中、出牌飞牌、结算动画、背景音乐、出牌音效与震动。
- 10. 发布与仓库：项目已配置 HTTPS 远程仓库、release 签名、runRelease 运行配置和敏感文件忽略规则；签名私钥只保存在本机，不上传 GitHub。

开屏背景素材来自 Wikimedia Commons 的 The Cloisters Playing Cards 图片，页面标注为 CC0，可自由复制、修改和分发。项目内对原图右上空位做了补牌修复，保留原素质质感，同时避免竖屏开屏出现右上角缺牌观感。应用图标使用 FreeIcons 的 Cards 成品素材作为主体，页面标注 Public Domain / CC0，项目对素材做了深绿牌桌底、金色边框和桌面小尺寸适配。

2. 阶段时间表

序号	项目阶段	项目开发内容	建模和规约文档内容	提交时间	汇报时间
1	开始 (Inception)	Android 开发框架、Java/Kotlin 开发、用例图 1、发牌/选牌/出牌用例	Use-Case Model、Vision、Supplementary Specification、Glossary、Iteration Plan	第 1 周周五晚	第 11 周
2	化迭代 1 (Elaboration) / 构造 (Construction)	用例图 1、用例实	Domain Models、System Sequence Diagrams、Operation Contracts、UML Package Diagrams、UML Interaction	第 2 周周五晚	第 12 周

			Diagrams、UML Class Diagrams		
3	化迭代 2 (Elaboration) / 构造 (Construction)	用例图 2、用例实现、 出牌规则模块、蓝牙/Wi Fi 通信模块	Applying GoF Design Patterns	第 3 周周五晚	第 13 周
4	化迭代 3 (Elaboration) / 构造 (Construction)	用例图 3、用例实现、 AI 算法模块	UML State Machine Diagrams and Modeling、refine the above models	第 4 周周五晚	第 14 周
5	交付 (Transition)	系统集成	最后资料：文档和集成 代码	第 5 周周五晚	第 15 周

3. 小组分工与协作方式

3.1 总体分工

成员	负责内容	主要交付物	答辩重点
Annakuliyev Sedar	UML 建模、ProcessOn 绘图、图形 布局和建模材料整理	用例图、类图、顺序图、状态图、活 动图、组件图、部署图，UML 建模说 明	UML 图含义、分析建模和设计建模关 系、图与代码对应
莫均杰	主体代码、架构设计、游戏规则、 AI、蓝牙、UI、Release 集成	Kotlin/Compose 源代码模块AI 策 略、蓝牙房间、玩家中心、Release APK/AAB	MVC 分层、GRASP/GoF 设计应用、 南北规则、多人蓝牙、AI 策略、发布 流程
郭铭涛	测试设计、自动化测试、模拟蓝牙测 试、运行验收、测试报告和录屏验证	单元测试、模拟测试、测试报告、手 动验收表、演示录屏	测试覆盖、测试方法、蓝牙风险、 Release 验收、录屏展示

3.2 段分工行情况

阶段	Annakuliyev Sedar	莫均杰	郭铭涛
开始	整理核心参与者和初版用例图	搭建 Android 项目环境，确定 Kotlin + Compose 技术路线	协助确认用例边界和基础验收场景
化迭代 1	绘制类图、顺序图、包图和交互关系	完成 MVC 分层建立 Model、 Controller、View 基构	准备牌型、首出、过牌等规则测试思 路
化迭代 2	更新蓝牙、规则相关 UML 元素	实现南北规则、牌型识别、蓝牙房间 和消息协议	编写规则测试、消息编解码测试、房 间座位测试
化迭代 3	绘制状态图，补充 AI、玩家成长与牌 局状态转换	实现三档 AI 策略、牌桌 UI 状态切换 和玩家成长系统	测试 AI 合法动作、随机多局完成、玩 家成长和动画 key 稳定性
交付	核对 UML 图与最终代码一致	集成音效、Release 安装包、GitHub 托管和最终文档	执行单元测试、构建测试、真机/模拟 器验收

3.3 UML 图与阶段产物总映射

本项目的 UML 图按照 UP 迭代阶段逐步形成并在交付阶段统一校对。开发者先使用 PlantUML 文本描述图结构，便于在 Android Studio 中预览、版本管理和随代码修改；随后将 PlantUML 内容导入或参照迁移到 ProcessOn UML，调整布局后作为正式报告图。下表把项目内 7 类 UML 图对应到 Excel 阶段、阶段产物和代码验证点。

UML 绘图过程中，开发者先把需求、源码包结构和关键调用流程整理为文字约束，再让 Codex/GPT-5 生成 PlantUML 草图，包括参与者、类关系、消息顺序、状态节点和组件依赖。随后在 Android Studio 或 PlantUML 预览中检查类名、方法名和箭头方向，最后将 PlantUML 内容导入或参照迁移到 ProcessOn UML 调整布局。这样既利用 AI 快速生成图形初稿，又保留人工对图、文、代码一致性的最终校验。

AI 辅助绘图的价值主要体现在三点：第一，能把分散在 model、controller、transport、profile、view 包中的类自动归纳为类图和组件图；第二，能根据 GameController.play、GameTransport.observeEvents、ProfileController.settleMatch 等调用链生成顺序图草稿；第三，能把 Splash、Lobby、ReadyToStart、Dealing、Playing、Paused、Result、HostDisconnected 等状态整理为状态图节点，减少手动画图时遗漏状态的风险。

UML 图	阶段归属	阶段产物定位	主要表达内容	对应代码/文档
用例图 use_case.png	阶段 1：开始	Use-Case Model、项目愿景和核心用例边界	玩家、房主、加入者、人机参与者，以及大厅、人机对局、好友蓝牙、规则设置、玩家中心、结算等用例	docs/requirements.md、view/ChudadiApp.kt
类图 class_diagram.png	阶段 2：细化迭代 1	Domain Model、Design Model、UML Class Diagram	Card、GameState、RuleSet、GameController、AiStrategy、GameTransport、ProfileController 等类和接口关系	model/、controller/、transport/、profile/
顺序图 sequence_game_turn.png	阶段 2 和阶段 3	System Sequence Diagram、Interaction Diagram	玩家出牌、Controller 校验、状态更新、蓝牙客户端请求、房主校验、广播快照、客户端断线托管和房主断线退出顺序	GameController.kt、GameMessage.kt、BluetoothTransports.kt
活动图 activity_play.png	阶段 3：细化迭代 2	用例实现流程、操作契约流程化表达	开局、发牌、选择牌、出牌/不出、非法出牌提示、蓝牙等待确认、结算和再来一局	PlayerActionPolicy.kt、ChudadiApp.kt
状态图 state_game.png	阶段 4：细化迭代 3	UML State Machine Diagram、状态建模	Splash、Lobby、Nearby、ReadyToStart、Dealing、Playing、Paused、Result、断线托管等状态转换	Screen、GameStartPhase、BluetoothEntryMode
组件图 component_diagram.png	阶段 2 和阶段 5	UML Package/Component Diagram、模块依赖说明	View、Controller、Model、Transport、Profile、Audio、Android Bluetooth API、SharedPreferences 等组件依赖	docs/design.md、源码包结构
部署图 deployment_diagram.png	阶段 3 和阶段 5	Deployment Model、联机和测试环境说明	房主手机、加入者手机、Classic Bluetooth RFCOMM、Android Studio、Gradle、真机/模拟器测试环境	BluetoothHostTransport、BluetoothClientTransport、release_testing.md

阶段产物关系可以概括为：开始阶段先确定“要做什么”，用例图作为需求入口；细化迭代 1 把用例转成领域对象、类关系和交互流程；细化迭代 2 将规则和蓝牙通信落地，用顺序图、活动图和部署图解释多人联机；细化迭代 3 用状态图约束 AI、玩家成长、动画、暂停和结算状态；交付阶段用组件图、部署图和 ProcessOn 汇总图证明最终系统结构、运行环境和文档一致。

4. 阶段 1：开始（Inception）

4.1 阶段目标

开始阶段的目标是完成开发环境配置、明确项目愿景、确定核心用例和基础迭代计划。根据实训启动纲要，项目需要实现 Android 锄大地扑克牌游戏，并体现 UML 建模、Android 开发、多人蓝牙连接、南北规则、AI 策略和 AI 工具使用。

4.2 开发内容

本阶段完成 Android Studio 和 Gradle 环境配置，建立 Kotlin + Jetpack Compose 项目基础，确认应用主入口、资源目录和构建脚本。项目入口为 `app/src/main/java/com/example/uml_chudadi/MainActivity.kt`，主界面集中在 `app/src/main/java/com/example/uml_chudadi/view/ChudadiApp.kt`。

需求分析时，将游戏拆分为以下核心用例：

- 1. 玩家启动 App 并进入大厅。
- 2. 玩家选择规则，包括北方规则和南方规则。
- 3. 玩家进入人机对局，选择简单、普通、困难 AI。
- 4. 玩家创建或加入好友蓝牙房间。
- 5. 房主添加人机补位并开始四人对局。
- 6. 玩家完成发牌、选牌、出牌、不出、提示、结算和再来一局。
- 7. 玩家进入玩家中心，查看等级、金币、战绩、成就和头像。

按照要求，本阶段没有直接堆叠大量界面，而是先确定项目边界和主流程：

分析项	具体结论	对应代码/文档
技术框架	采用 Android Studio + Gradle + Kotlin + Jetpack Compose；项目使用 Kotlin 作为 Android 官方推荐语言	<code>settings.gradle.kts</code> 、 <code>build.gradle.kts</code> 、 <code>MainActivity.kt</code>
主入口	单 Activity 架构， <code>MainActivity</code> 只负责 Compose 内容，具体页面逻辑交给 <code>ChudadiApp</code>	<code>MainActivity.kt</code> 、 <code>view/ChudadiApp.kt</code>
游戏对象	玩家、扑克牌、牌堆、牌型、规则、牌局状态、出牌动作、蓝牙房间、玩家档案	<code>model/</code> 、 <code>transport/</code> 、 <code>profile/</code>
核心用例	开屏、进入大厅、选择规则、人机对局、蓝牙房间、发牌、选牌、出牌、不出、提示、结算、再来一局	<code>docs/uml/use_case.puml</code>
质量边界	不新增不必要第三方依赖，优先使用 Android/Compose/蓝牙原生 API，降低构建风险	<code>docs/requirements.md</code>

4.3 建模与规约内容

Use-Case Model、Vision、Supplementary Specification、Glossary、Iteration Plan 是开始阶段需要覆盖的规约内容。本项目没有把这些内容拆成五个独立文件，而是整合在需求分析、设计说明、UML 图、本报告和阶段时间表中，具体对应关系如下：

规约项	本项目体现	对应材料
Use-Case Model	描述玩家、房主、加入者和人机参与者，以及开屏、大厅、人机、蓝牙、规则、牌局、结算等用例	<code>docs/uml/use_case.puml</code> 、 <code>docs/uml/use_case.png</code>

Vision	明确项目定位为 Android 锄大地棋牌游戏，交付目标包括功能、UML、测试和 Release 包	docs/requirements.md、本报告第 1 章
Supplementary Specification	记录 MVC 分层、多人蓝牙、AI 策略、南北规则、测试覆盖和发布包等非功能要求	docs/requirements.md、docs/design.md
Glossary	统一解释发牌、首出、压牌、过牌、南北规则、房主权威同步、玩家成长等术语	docs/complete_project_documentation.md、本概相关章节
Iteration Plan	依据 Excel 时间节点规划开始、三次细化/构造迭代和交付阶段	本概第 2 章“阶段时间表”、docs/scrum.md

项目定位为“Android 锄大地棋牌游戏”，不是单纯演示界面。非功能要求包括 MVC 分层、可测试的规则模型、多人蓝牙 C/S 房间、GoF 策略模式实现的 AI 模块、发布包可安装、文档与 UML 可追踪。

4.4 实现成果

项目完成 Android 工程搭建，确定了 Kotlin + Jetpack Compose 的实现路线，并在 docs/requirements.md 中形成需求分析、用例边界和非功能要求。需求梳理阶段借助 Codex/GPT-5 阅读实训纲要和时间节点，开发者结合 Android Studio 工程范围对功能边界进行校对。UML 建模从用例图开始，为后续领域模型、类图和状态图提供基础。

开始阶段的 AI 使用集中在方案设计和需求拆解。开发者把实训纲要、Excel 迭代节点和预期演示目标输入 Codex/GPT-5，由模型辅助提炼出“可运行 Android 游戏、七类 UML 图、MVC 分层、多人蓝牙、南北规则、三档 AI、测试和 Release 交付”等主线。人工确认时没有直接照搬模型输出，而是结合 Android Studio 模板项目、Gradle 依赖风险和真机蓝牙测试条件，决定不新增第三方依赖，优先使用 Kotlin、Compose、SharedPreferences 和 Android Classic Bluetooth 原生 API。

这一阶段还借助 AI 讨论了技术选型：界面层选择 Compose 是因为单 Activity 项目可以用状态驱动页面切换，规则与 AI 模块选择纯 Kotlin 是为了便于本地单元测试，蓝牙通信选择 C/S 房主房间是为了满足多人联机并避免多端各自计算导致状态分叉。AI 给出多个方案后，开发者保留了最适合课程验收和本地可运行环境的实现路线。

本阶段使用的 GRASP 原则和 GoF 设计模式如下：

GRASP/GoF 设计应用	阶段作用	代码或文档落点
GRASP 控制器（Controller）	在需求阶段先确定由 Controller 接收 UI 意图并协调 Model，避免后续把规则和 UI 写在一起	docs/design.md、GameController.kt、RoomController.kt、ProfileController.kt
GRASP 高内聚/低耦合（High Cohesion / Low Coupling）	将规则、AI、蓝牙、玩家成长、音频和界面拆分为独立职责，降低互相修改影响	model/、controller/、transport/、profile/、view/、audio/
GRASP 防变化/间接性（Protected Variations / Indirection）	先确定 Screen、RuleSet、GameTransport 等边界，为后续页面、规则、蓝牙和牌桌切换做基础	Screen、RuleSet、GameTransport、用例图、状态图

阶段成果展示：

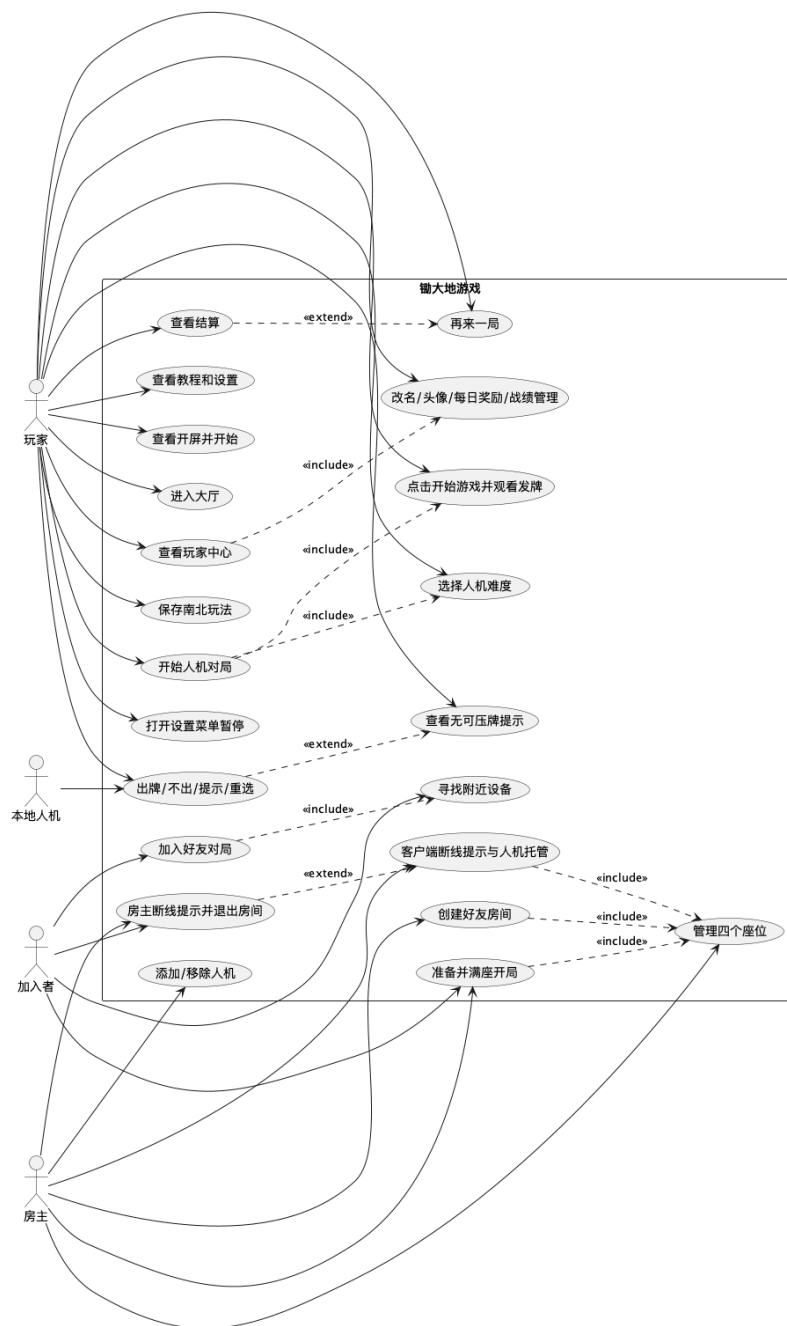
成果类型	展示内容	证据材料
Code	单 Activity + Compose 主入口，启动后加载 ChudadiApp()	app/src/main/java/com/example/uml_chudadi/MainActivity.kt

Text	项目目标、功能需求、非功能需求和要求点映射	docs/requirements.md
Image	用例图展示玩家、房主、加入者、人机和主要用例	docs/uml/use_case.png

主入口代码片段如下：

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Uml_chudadiTheme {
                ChudadiApp()
            }
        }
    }
}
```

阶段图片成果如下：



阶段 1 用例图

4.5 问题与解决

问题	解决方法
锄大地规则存在南北差异	在需求阶段将规则拆分为可配置的 RuleSet，后续由北方规则和南方规则分别实现
多人蓝牙可能复杂且真机环境有限	先确定房主 C/S 拓扑，并计划使用本地模拟传输补充测试
课程要求既包含代码又包含建模和文档	在阶段计划中将 UML、代码、测试和 AI 使用说明同时纳入交付

5. 阶段 2：细化迭代 1 / 构造（Elaboration / Construction）

5.1 阶段目标

本阶段目标是完成面向对象分析和初步设计，建立项目的领域模型、系统顺序图、操作契约、包图、交互图和类图，并实现核心代码结构。

5.2 面向对象分析

锄大地游戏的核心对象包括扑克牌、牌堆、玩家、牌型、规则、牌局状态、玩家动作和联机消息。根据这些对象，项目将代码分为 Model、Controller、View、Transport、Profile 和 Audio 等模块。

模块	关类	职责
牌与牌型	Card、Deck、HandClassifier、HandType	表示牌面、发牌、牌型识别和牌型比较基础
规则	RuleProfile、NorthRuleSet、SouthRuleSet	表示南北玩法、首出规则和压牌规则
回合控制	GameController、GameState、Move	处理出牌、不出、三家过牌重置和胜负结算
AI	AiStrategy、NeuralAiStrategy、NeuralAiProfile、NeuralMoveFeatures、NeuralMoveModel	表示三档神经网络人机策略、特征编码和端侧模型推理
联机	GameTransport、BluetoothHostTransport、BluetoothClientTransport、LocalRoomTransport	隔离真实蓝牙和本地模拟传输
玩家成长	PlayerProfile、ProfileController、ProfileStore	管理金币、等级、经验、战绩、头像和成就
UI 与反馈	ChudadiApp、CardRoomMusicPlayer、TableAnimationKeys	负责页面、动画、音乐、音效和交互反馈

分析建模阶段重点把“现实牌局中的名词”转换为“可维护的领域对象”：

1. Card 表示单张牌，内部由 Suit 和 Rank 组成；Deck 负责生成 52 张牌、洗牌和按 seed 发牌。
 2. HandClassifier 只负责识别牌型，不关心 UI 和蓝牙；这样规则、AI、测试都能复用同一套牌型判断。
 3. RuleSet 负责规则差异，NorthRuleSet 和 SouthRuleSet 只改变首出牌、炸弹增强、同张数压制等规则点，不复制整套游戏流程。
 4. GameState 是牌局唯一状态源，保存玩家手牌、当前回合、上一手、过牌次数、胜者、提示消息和规则。
 5. Move.Play 与 Move.Pass 表示动作命令，玩家、AI、蓝牙客户端最终都转换成同一种动作模型。
 6. RoomSeat 表示蓝牙房间座位，既能表达真人、房主、人机、空位，也能表达在线、断线、托管和准备状态。
 7. PlayerProfile 表示本地玩家档案，包含昵称、金币、等级经验、头像、战绩和成就。
- 这些对象对应类图中的核心区域：model 是牌局领域模型，controller 是业务规则执行者，transport 是联机协议和传输，profile 是玩家成长数据，view/audio 是页面、动画和反馈。

5.3 MVC 分层设计

项目采用 MVC 思想组织代码：

1. Model 层负责领域数据和规则，包括 Card、Deck、HandType、RuleSet、GameState、Player、RoomSeat、GameSnapshot 和 PlayerProfile。

2. Controller 层负责业务逻辑，包括 GameController、PlayerActionPolicy、RoomController、NetworkMoveGuard 和 ProfileController。
3. View 层使用 Jetpack Compose 实现界面展示，包括开屏、大厅、规则设置、人机难度选择、蓝牙房间、牌桌、结算和玩家中心。

这种分层让 UI 不直接处理复杂规则，让规则、AI、蓝牙和测试能够独立演进。

更细的实现边界如下：

层次	代码位置	主要类	具体职责
Model	app/src/main/java/com/example/uml_chudadi/model	Card、Deck、HandType、GameState、RuleSet、Player	表示牌、牌型、玩家、规则和牌局状态，不依赖 Android UI
Controller	app/src/main/java/com/example/uml_chudadi/controller	GameController、AiStrategy、PlayerActionPolicy、NetworkMoveGuard	处理发牌、出牌、过牌、AI 决策、按钮可用性和网络请求校验
Transport	app/src/main/java/com/example/uml_chudadi/transport	GameTransport、BluetoothHostTransport、BluetoothClientTransport、GameMessageCodec	隔离蓝牙 socket、消息编解码、快照同步和房间座位
Profile	app/src/main/java/com/example/uml_chudadi/profile	PlayerProfile、ProfileController、ProfileStore	处理等级、金币、头像、战绩、成就、每日奖励和本地保存
View	app/src/main/java/com/example/uml_chudadi/view	ChudadiApp、SplashScreen、LobbyScreen、GameScreen	负责 Compose 页面、动画、弹窗和用户交互
Audio	app/src/main/java/com/example/uml_chudadi/audio	CardRoomMusicPlayer、GameSoundEffects	负责背景音乐、出牌音效和震动反馈

例如玩家点击 出牌 时，View 层只收集选中的 Card；PlayerActionPolicy 判断按钮是否可用；GameController 调用 RuleSet 校验是否合法；成功后生成新的 GameState；如果是蓝牙对局，房主再通过 GameSnapshot 广播给客户端。这个流程体现了 MVC 的分层和顺序图中的调用链。

5.4 建模成果

本阶段重点完成类图、顺序图、包图和交互图。类图展示 Model、Controller、Transport、Profile 和 View/Audio 的类关系；顺序图展示玩家一次本地出牌以及蓝牙房主权威同步的调用流程。

UML 相关材料位于 docs/uml/：

UML 图	文件	说明
用例图	docs/uml/use_case.png	玩家、房主、加入者和人机与核心用例
类图	docs/uml/class_diagram.png	Model、Controller、Transport、View/Audio 类关系
顺序图	docs/uml/sequence_game_turn.png	本地出牌和蓝牙同步调用顺序
活动图	docs/uml/activity_play.png	开局、出牌、不出、结算和再开局流程
状态图	docs/uml/state_game.png	页面、房间、牌桌、暂停和结算状态
组件图	docs/uml/component_diagram.png	UI、控制器、规则、消息、蓝牙和系统能力
部署图	docs/uml/deployment_diagram.png	房主手机、加入者手机、蓝牙栈和开发测试环境

5.5 实现成果

本阶段完成了 MVC 工程结构和主要模型设计。Model 层负责 Card、Player、GameState、RuleSet、HandType 等领域数据和规则抽象；Controller 层负责 GameController、AiController、RoomController、PlayerActionPolicy 等业务流程；View 层通过 ChudadiApp 和 Compose 组件展示状态并收集用户意图。GameState 仍是牌桌状态源，但它属于 MVC 中的 Model，不直接处理界面和蓝牙线程。领域对象和 UML 草图在 Codex/GPT-5 辅助下从用例流程中提取，开发者再依据源码包结构、PlantUML 预览和 ProcessOn 图进行校对。

代码实现亮点之一是把牌局状态集中在 GameState，并通过 Kotlin data class 的 copy 方式生成新状态。出牌、过牌、AI 行动和蓝牙同步都围绕 GameState 变化展开，Compose 页面只根据状态重组，不直接修改手牌列表。这样可以减少 UI 层和规则层各自维护变量造成的不一致。

第二个亮点是把用户动作封装为 Move.Play 和 Move.Pass。无论动作来自底部手牌按钮、AI 策略，还是蓝牙客户端发送的 MOVE_REQUEST，最终都会进入同一套 GameController 校验流程。这个设计让规则测试、AI 测试和网络测试复用同一条业务路径，也方便在答辩时说明“玩家、人机、远端玩家”并不是三套逻辑。

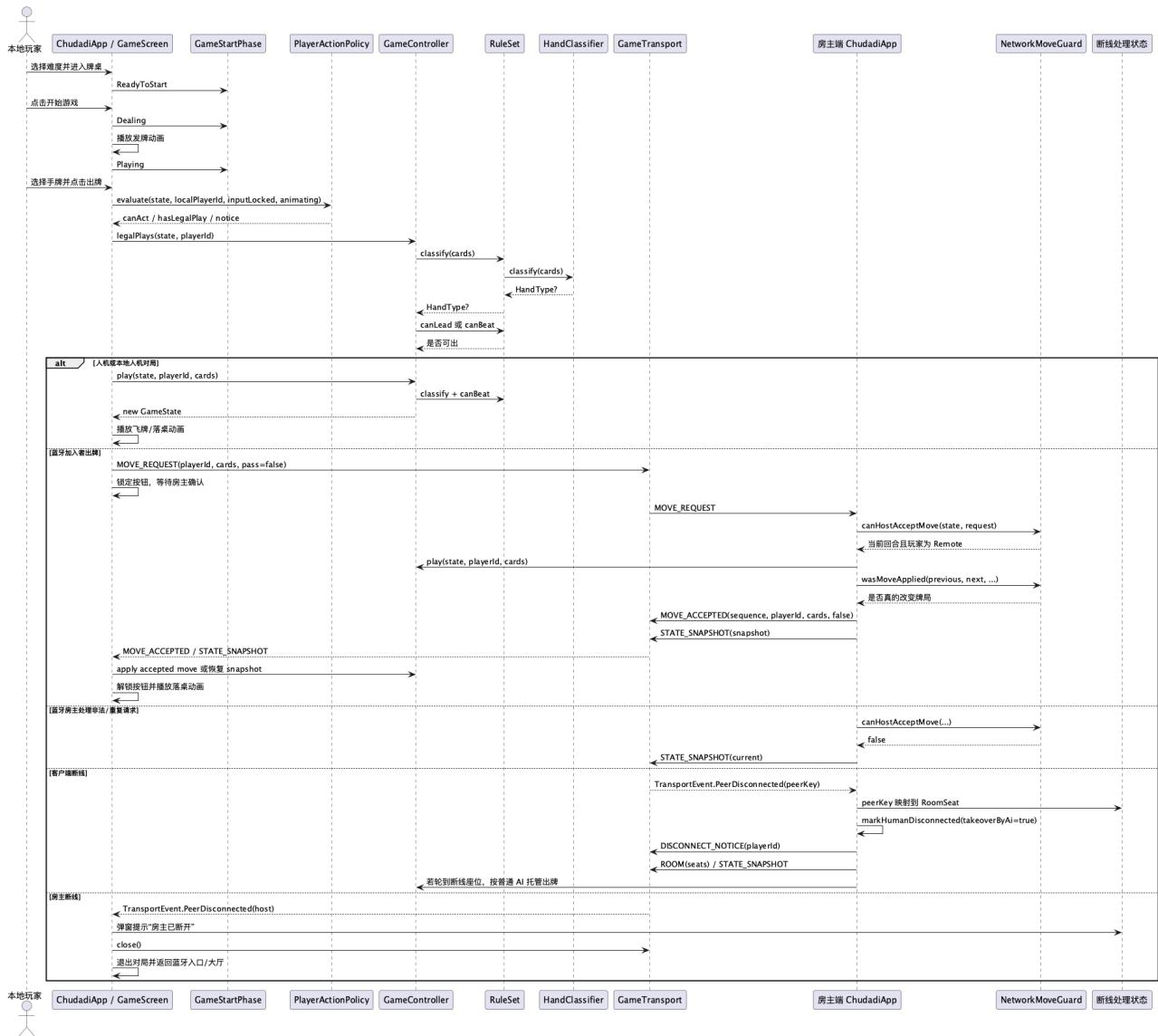
本阶段的 AI 辅助编码方式比较具体：开发者把领域对象清单、包名和 Kotlin 约束提供给 Codex/GPT-5，模型生成 Card、Deck、RuleSet、GameState、Move 等数据结构和接口初稿；随后开发者在 Android Studio 中编译，依据 Kotlin 报错和运行问题继续让 AI 修正类型、导入、空值和包路径问题。最终代码以本地编译和单元测试通过为准，而不是以模型文本为准。

阶段成果展示：

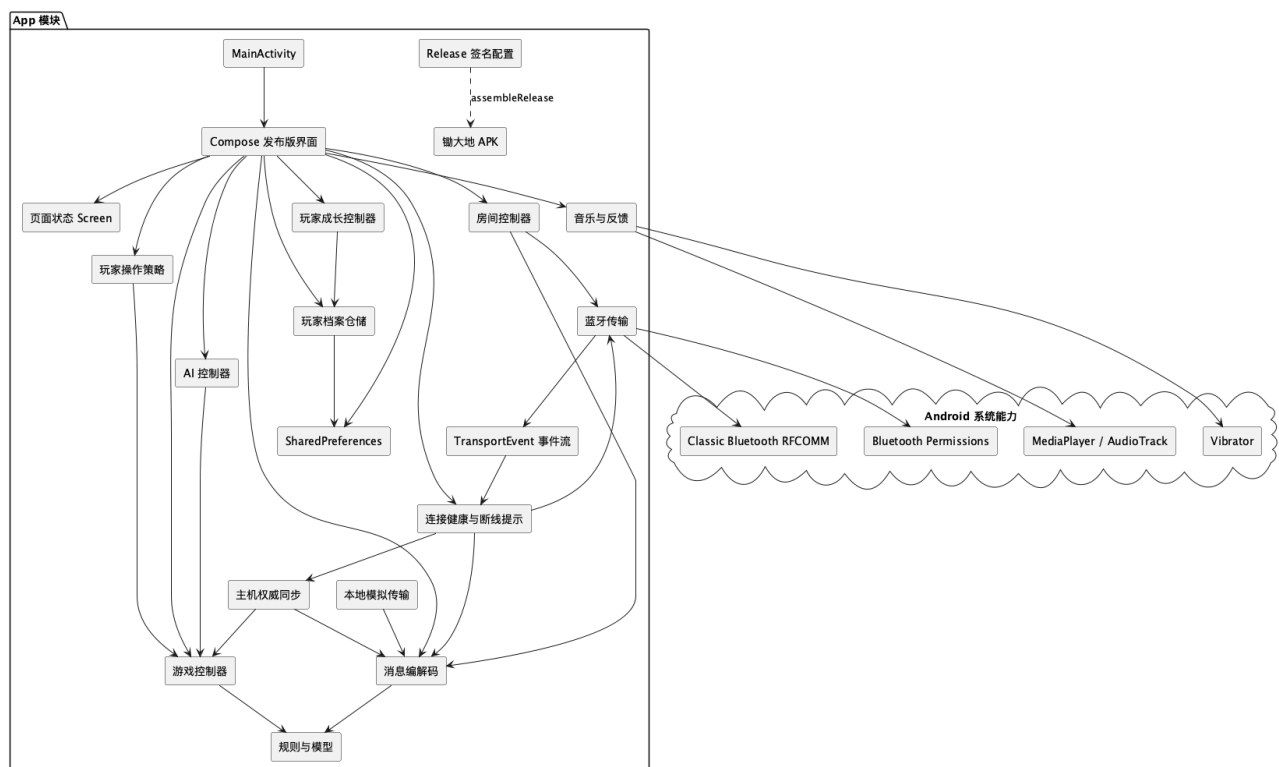
成果类型	展示内容	证据材料
MVC	Model 层保存牌、玩家、规则和牌局状态；Controller 层推进出牌、过牌、AI 和房间流程；View 层只展示状态和触发用户意图	model/, controller/, view/ ChudadiApp.kt
设成果	MVC 分层边界明确，Model、Controller、Transport、Profile、View、Audio 职责分离	docs/design.md
Image	类图展示 Model、Controller、Transport、Profile、View/Audio 的依系	docs/uml/class_diagram.png
交互成果	顺序图展示本地出牌和蓝牙房主权威同步调用链	docs/uml/sequence_game_turn.png

本阶段使用的 GRASP 原则和 GoF 设计模式如下：

GRASP/GoF 设计应用	阶段作用	代码或文档落点
GRASP 控制器（Controller）	GameController、RoomController、AiController、PlayerActionPolicy 接收系统操作并协调 Model，不把业务流程写入 Compose 页面	controller/
GRASP 信息家（Information Expert）	Card、HandType、RuleSet、GameState 等对象持有完成牌型识别、规则判断和状态推进所需的信息	model/
GoF 策略模式（Strategy）	AiStrategy 和 RuleSet 通过接口封装可替换算法；三档 AI 由 NeuralAiProfile 配置同一神经网络模型	AiStrategy.kt、NeuralAiStrategy.kt、RuleSet.kt



阶段2 顺序图



5.6 问题与解决

问题	解决方法
UI 直接处理规则会导致代码复杂	将 删除 下沉到 RuleSet 和 GameController，Compose 只收集用户意图
牌型识别需要被规则、AI、测试复用	独立设计 HandClassifier，避免在多个模块重复实
UML 容易与代码不同步	先用 PlantUML 文本记录结构，再迁移到 ProcessOn 并按最终代码校对

6. 阶段 3：细化迭代 2 / 构造（Elaboration / Construction）

6.1 阶段目标

本阶段完成出牌规则模块和多人通信模块，并在代码实现中应用 GRASP 原则与 GoF 设计模式。MVC 分层架构用于组织 Model、Controller、View 的职责边界；规则模块采用 GoF 策略模式，蓝牙传输采用 GoF 适配器模式，传输事件采用 GoF 观察者模式；GameController、RoomController、NetworkMoveGuard 体现 GRASP 控制器和防变化原则；GameMessageCodec、ProfileStore 等独立服务对象体现 GRASP 纯虚构原则。房主权威同步、断线托管和状态快照属于联机架构机制，不作为设计模式命名。

6.2 出牌规则模块

项目使用 RuleSet 表示规则接口，并通过 NorthRuleSet 和 SouthRuleSet 适配南北玩法差异：

规则项	北方规则	南方规则
首出牌	黑桃 3 首出	方块 3 首出
五张牌比较	四带一、同花顺可作为增强强牌跨牌型压制普通牌	五张牌之间按顺子、同花、葫芦、四带一、同花顺等级比较
压牌限制	根据北方增强牌规则判断	更强调同张数压制和五张牌等级
配置方式	通过规则设置保存到本机	通过规则设置保存到本机

牌型识别由 HandClassifier 完成，支持单张、对子、三张、顺子、同花、葫芦、四带一和同花顺。GameController 在玩家出牌时调用规则模块进行合法性校验，并根据三家过牌、上一手牌和当前玩家推进牌局状态。

规则模块的详细执行流程如下：

- 1. 玩家或 AI 选择若干张牌后，GameController.play() 首先判断是否轮到该玩家。
- 2. 控制器调用 state.ruleSet.classify(cards) 识别牌型；无法识别时返回非法提示。
- 3. 如果是首出，调用 state.ruleSet.canLead(cards, state.firstTurn) 检查是否包含规则指定首牌。
- 4. 如果是跟牌，调用 state.ruleSet.canBeat(previous, next) 判断是否可以压过上一手。
- 5. 合法出牌会从玩家手牌中移除对应牌，刷新 lastMove、currentPlayerId 和提示消息。
- 6. 三家连续不出后，上一手清空，下一位玩家获得自由出牌权。
- 7. 某玩家手牌为 0 时，winnerId 产生，进入结算。

南北规则的设计不是把整套流程复制两遍，而是把差异封装在 RuleProfile 和 RuleSet 内。这样 UI、AI、蓝牙都只依赖规则接口，不需要知道当前是北方还是南方玩法。蓝牙开局时，房主把规则名放入 START 消息，客户端用本地 ruleSetByIdOrName(ruleName) 转换为同一规则对象，保证多端比较结果一致。

6.3 蓝牙通信模块

多人联机采用房主权威的 C/S 模式，而不是一对一蓝牙匹配。房主手机创建房间并监听连接，加入者手机作为客户端连接房主。房间固定四个座位，房主可以添加人机补位，满四座并准备后开始游戏。

蓝牙通信的核心流程：

1. 房主创建房间，建立 BluetoothHostTransport，持续监听加入者连接。
2. 加入者搜索附近或已配对设备，并通过 BluetoothClientTransport 连接房主。
3. 加入者发送 HELLO，其中包含昵称和客户端标识等基础信息。
4. 房主维护四座位 RoomSeat，记录 SeatConnectionState、clientId、设备地址和是否由 AI 托管。
5. 房主广播 ROOM(roomId, hostEpoch, seats)，使所有客户端看到一致座位。
6. 开局时房主生成 seed、同步规则并广播 START 消息。
7. 客户端只发送 MOVE_REQUEST，不直接修改本地牌局。
8. 房主使用 NetworkMoveGuard 判断是否为当前玩家的有效请求。
9. 房主调用 GameController 校验和推进状态。
10. 房主广播 MOVE_ACCEPTED 和 STATE_SNAPSHOT，所有客户端以房主快照为准刷新牌桌。
11. 如果客户端，输发出 TransportEvent.PeerDisconnected，房主把对应座位标记为“断线，人机托管中”，并广播 DISCONNECT_NOTICE。
12. 当前发布版没有实现自动断线重连；断线客户端需要重新进入蓝牙流程。
13. 如果房主断线，客户端弹窗提示连接断开，关闭当前传输并退出房间，提示重新创建或重新加入。

这种方式可以避免多台手机各自结算导致的状态分叉，也便于测试时用 LocalRoomTransport 和 SimulatedBluetoothHubTest 模拟多人房间。由于房主是权威端，当前版本对房主断线采用保守处理：直接提示并退出，而不尝试新房主迁移，避免恢复过程中出现多端状态不一致。

传输层和协议层的最新实现细节：

类型	代码落点	说明
传输事件	TransportEvent.Message、PeerConnected、PeerDisconnected、Error	socket 线程不再只回调字符串，而是能区分收到消息、连接、断开和错误
定向发送	GameTransport.sendTo(peerKey, message)	房主可以只回复某个客户端，例如房满错误或同步快照
房间标识	roomId	区分不同房间和再开局，避免旧消息混入新房间
房主代	hostEpoch	协议中保留的版本字段，用于过滤旧快照或后续扩展
客户端标识	clientId	用于标记加入者身份和座位信息，当前版本不承诺自动断线恢复
连接状态	SeatConnectionState	座位可显示在线、断线、空位等状态
AI 托管	takeoverByAi	真人断线时座位不删除，由房主 AI 继续托管

新增或扩展的消息包括：

消息	用途
DISCONNECT_NOTICE	通知某座位断线并已进入 AI 托管
HEARTBEAT	协议预留的连接健康消息，当前主要依靠 socket 断开和错误事件提示
HOST_MIGRATION、REJOIN_REQUEST / REJOIN_ACCEPTED	协议模型中保留过扩展入口，但当前发布版不承诺自动房主迁移或断线重连
STATE_SNAPSHOT	完整牌局快照，包含 roomId、hostEpoch、玩家、手牌、桌面和提示

客户端断线时，房主通过 `TransportEvent.PeerDisconnected(peerKey)` 定位座位，调用 `markHumanDisconnected(takeoverByAi = true)` 保留座位。UI 显示“某玩家断线，已由人机托管”，如果轮到该座位，由房主运行普通难度 AI 出牌并继续广播快照。当前版本没有实现自动断线重连，断线客户端需要重新进入蓝牙流程。

房主断线时，客户端弹窗提示房主连接已断开，暂停并退出当前对局。因为房主端保存权威牌局状态，当前版本不做新房主迁移，恢复方式是重新创建房间或重新加入。

6.4 本阶段 GRASP 与 GoF 设计应用

GRASP/GoF 设计应用	代码落点	解决的问题
GoF 策略模式 (Strategy)	RuleSet、AiStrategy、NeuralAiStrategy、NeuralAiProfile	南北规则和三档神经网络 AI 可替换，控制器只依赖接口
GoF 适配器模式 (Adapter)	GameTransport、BluetoothHostTransport、BluetoothClientTransport、LocalRoomTransport	统一真实蓝牙和模拟传输；UI/房间不依赖 BluetoothSocket
GoF 命令模式 (Command)	Move.Play、Move.Pass、MoveRequest	玩家、人机和远端动作统一表达，便于校验和转发
GoF 观察者模式 (Observer)	TransportEvent.Message、PeerConnected、PeerDisconnected、Error、observeEvents(listener)	传输层把消息、连接、断开和错误通知给房间与界面
GoF 状态模式 (State)	Screen、BluetoothEntryMode、GameStartPhase	页面、蓝牙入口和开局阶段由明确状态控制
GRASP 控制器 (Controller)	GameController、RoomController、NetworkMoveGuard	控制对象协调出牌、房间、请求校验和状态广播
GRASP 间接性/防变化	GameTransport、GameMessageCodec、GameSnapshot	隔离蓝牙 API、消息格式和同步变化
GRASP 构	GameMessageCodec、ProfileStore	协议编解码和本地保存拆成独立服务对象
GRASP 信息专家	RuleSet、HandClassifier、GameState	规则判断、牌型识别和状态推进由掌握信息的对象完成
联机机制说明 (非 GRASP/GoF 模式)	房主权威同步、客户端断线 AI 托管、房主断线弹窗退出	这些属于蓝牙可靠性机制，不作为 GRASP/GoF 设计模式命名

这些 GRASP 与 GoF 设计应用对应的技术讲解点如下：

- 策略模式是 AI 模块的核心设计之一。GameController 不知道具体 AI 算法，只接收 `AiStrategy.chooseMove(...)` 的结果；简单、普通、困难只是不同策略实现。
- 适配器思想体现在 GameTransport，UI 和房间逻辑面对的是统一接口，真实蓝牙和模拟蓝牙都能接入。
- GRASP 控制器与防变化原则用于联机校验。客户端不直接修改牌局，只发 `MOVE_REQUEST`；房主过滤重复、过期、非当前玩家请求后再广播权威状态，房主权威同步本身属于联机架构机制。

- 4. GRASP 纯虚构原则用于 GameMessageCodec。协议编解码集中在独立对象中，避免 UI 或 socket 线程散落解析逻辑。
- 5. GRASP 纯虚构与信息专家原则用于 ProfileStore。它集中处理 SharedPreferences，昵称、金币、头像、战绩、成就不直接由页面拼字符串保存。
- 6. GoF 观察者模式和 GRASP 防变化原则用于蓝牙稳定性改造。TransportEvent 将消息、连接、断开和错误转为可观察事件；客户端断线托管和房主断线退出是基于这些事件实现的业务处理机制。

6.5 实现成果

本阶段完成南北规则模块、牌型识别、联机消息协议、四座位房间、房主广播同步、本地模拟传输，以及蓝牙断线处理机制。传输层通过 TransportEvent 上报连接和断开，客户端断线时房主保留座位并让 AI 托管，房主断线时客户端弹窗提示并退出房间。规则差异、协议文本和模拟测试用例在 Codex/GPT-5 辅助下形成初稿，开发者通过 Gradle 测试和真机蓝牙验收校正，保证项目不是一对一蓝牙匹配，而是多人房间联机。规则模块的实现亮点在于 HandClassifier 与 RuleSet 分离。HandClassifier 专门识别单张、对子、三张、顺子、同花、葫芦、四带一、同花顺等牌型；NorthRuleSet 和 SouthRuleSet 再决定首出牌、五张牌跨牌型压制和同张数压制差异。这样后续如果老师要求调整南北玩法，只需要修改规则对象，不需要重写发牌、出牌、过牌和结算流程。

蓝牙模块的实现亮点是把 Android BluetoothSocket 的读写细节封装到 GameTransport 和 TransportEvent 中。UI 和房间逻辑不直接处理 socket 字符串，而是接收 Message、PeerConnected、PeerDisconnected、Error 等业务事件；房主端通过 sendTo(peerKey, message) 支持定向回复，通过 ROOM、START、MOVE_ACCEPTED、STATE_SNAPSHOT 等消息广播权威状态。

协议实现还加入了 URL encode/decode 处理，避免中文昵称或分隔符破坏 ROOM、START 等文本帧。GameMessageCodecTest 覆盖消息往返，SimulatedBluetoothHubTest 覆盖三人同时加入、房满拒绝、移动请求和客户端断线托管，用模拟测试弥补多台真机长期占用困难的问题。

调试阶段大量使用 AI 辅助修复 bug。开发者把 Android Studio Build Output、Logcat、真机复现步骤和蓝牙页面异常现象提供给 Codex/GPT-5，模型帮助定位“客户端加入后误显示为房主”“连接断开提示错误对象”“加入房间后立即断开”“房主断线后客户端提示不明确”等问题的原因。修复时并不是只改界面文案，而是同时调整座位状态、TransportEvent 分发、房主/客户端分支和弹窗恢复逻辑，保证错误提示与实际连接状态一致。

阶段成果展示：

成果类型	展示内容	证据材料
Code	RuleSet、NorthRuleSet、SouthRuleSet 封装南北规则差异	app/src/main/java/com/example/uml_chudadi/model/RuleSet.kt
Code	GameTransport 与 TransportEvent 把蓝牙消息、连接、断开、错误统一为事件	app/src/main/java/com/example/uml_chudadi/transport/GameTransport.kt
成果	DISCONNECT_NOTICE、STATE_SNAPSHOT 等消息支持断线提示、托管和状态同步；迁移/重连消息仅作为后续扩展入口保留	app/src/main/java/com/example/uml_chudadi/transport/GameMessage.kt

Image	顺序图展示客户端发请求、房主校验、广播快照的权威同步流程	docs/uml/sequence_game_turn.png
成果	模拟蓝牙覆盖三人加入、房满拒绝、房主广播、移动请求和客户端断线托管	app/src/test/java/com/example/uml_chudadi/transport/SimulatedBluetoothHubTest.kt

南北规则代码片段如下：

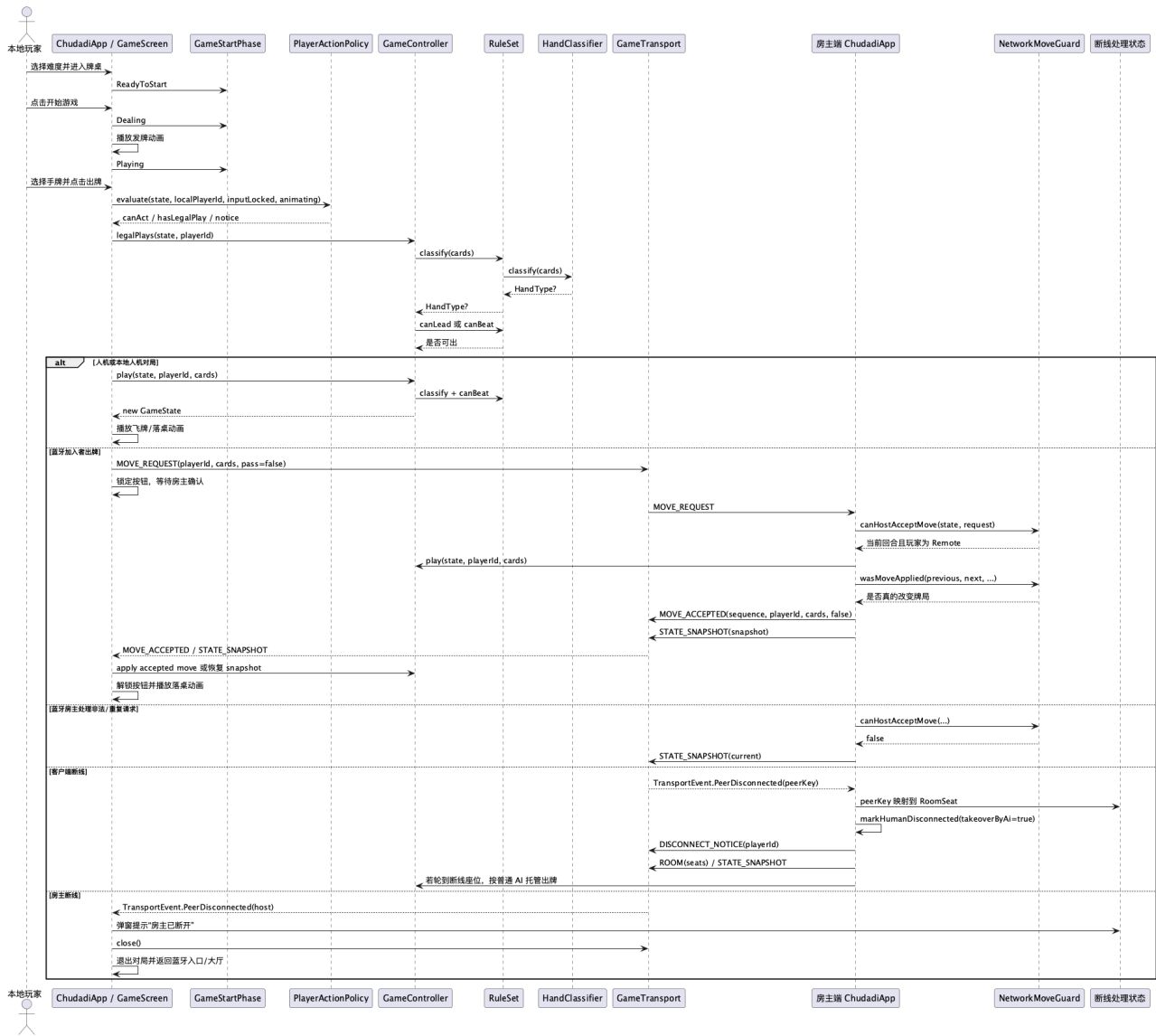
```
object NorthRuleSet : RuleSet {
    override val profile = RuleProfile(
        id = "north",
        title = "北方规则",
        firstCard = Card(Suit.Spades, Rank.Three),
        bombMode = BombMode.Enhanced,
        fiveCardRanking = true,
        sameSizeOnly = false
    )
}

object SouthRuleSet : RuleSet {
    override val profile = RuleProfile(
        id = "south",
        title = "南方规则",
        firstCard = Card(Suit.Diamonds, Rank.Three),
        bombMode = BombMode.None,
        fiveCardRanking = true,
        sameSizeOnly = true
    )
}
```

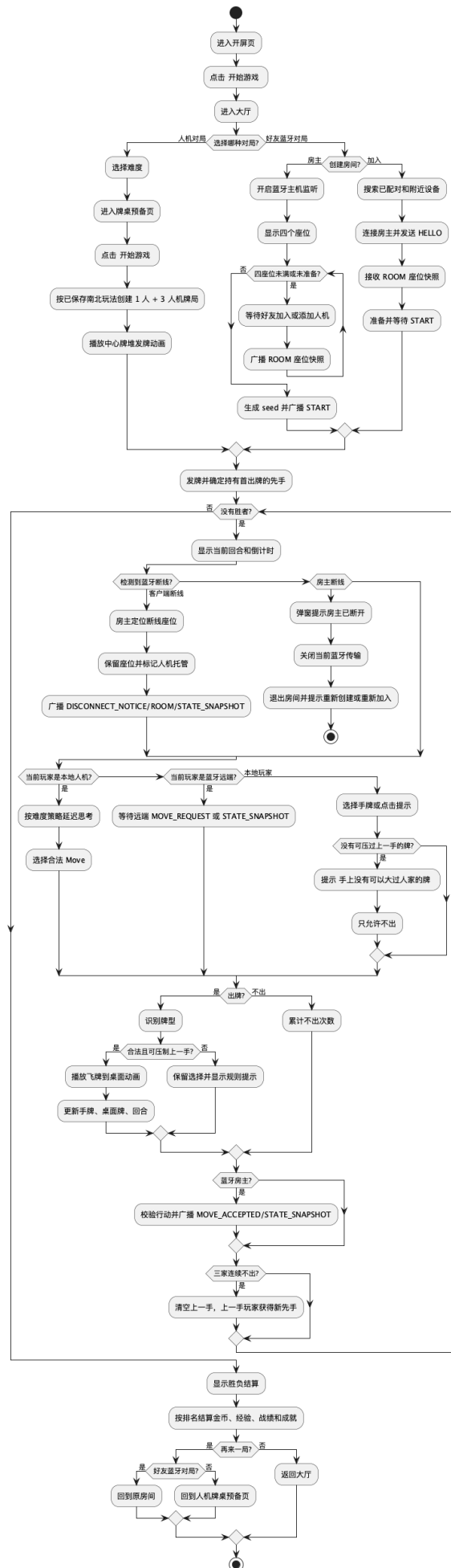
蓝牙传输抽象代码片段如下：

```
sealed class TransportEvent {
    data class Message(val raw: String, val peerKey: String? = null) : TransportEvent()
    data class PeerConnected(val peerKey: String, val deviceName: String = "") : TransportEvent()
    data class PeerDisconnected(val peerKey: String, val reason: String = "连接已断开") :
TransportEvent()
    data class Error(val reason: String, val peerKey: String? = null) : TransportEvent()
}
```

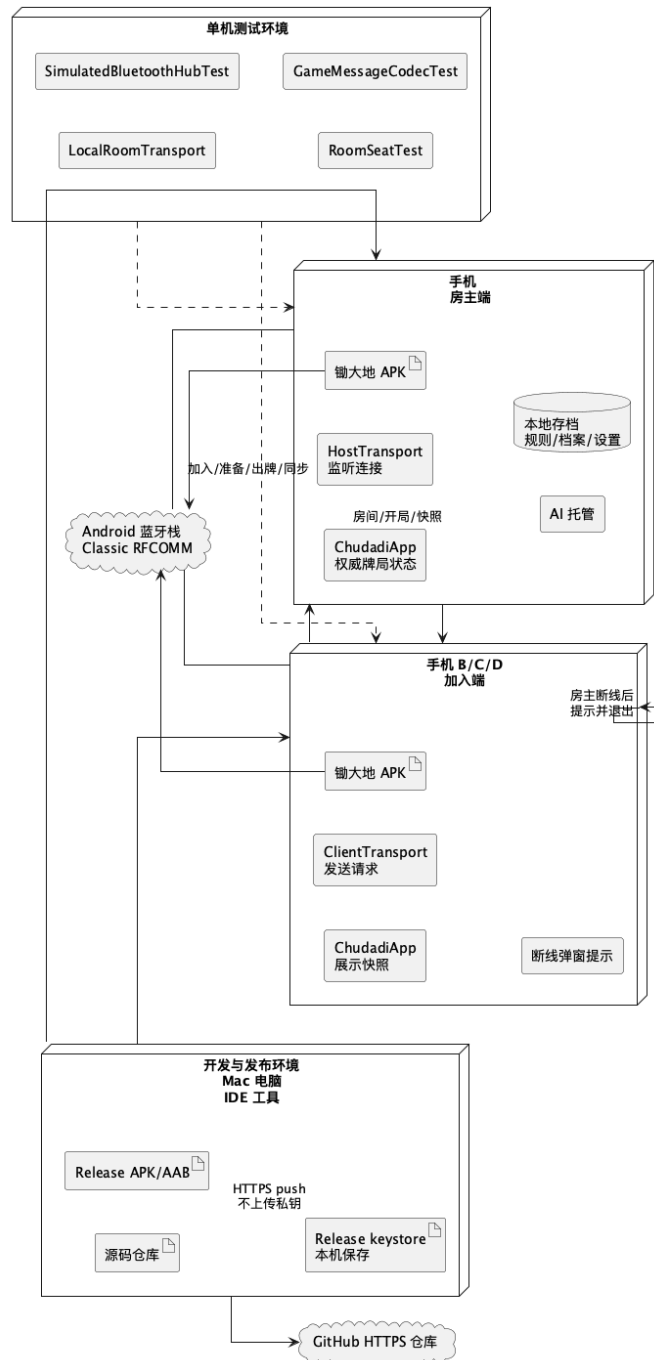
阶段图片成果如下：



阶段3 顺序图



阶段 3 活动图



阶段 3 部署图

6.6 问题与解决

问题	解决方法
蓝牙多机连接容易出现状态不一致	使用房主权威同步，客户端只提交请求，最终以房主快照为准
真机数量不足会影响联机测试	使用 <code>SimulatedBluetoothHubTest</code> 覆盖三人加入、房满拒绝、广播、移动请求和客户端断线托管
客户端断线会导致座位空缺和牌局中断	保留原座位并设置 <code>takeoverByAi</code> ，房主用普通 AI 继续代打；当前版本不承诺自动重连恢复
房主断线会导致权威端丢失	当前版本弹窗提示后退出房间，要求重新创建或重新加入，避免无权威端时状态分叉
南北规则差异会影响首出和压牌	使用 <code>RuleProfile</code> 、 <code>NorthRuleSet</code> 、 <code>SouthRuleSet</code> 拆分规则

7. 阶段 4：细化迭代 3 / 构造（Elaboration / Construction）

7.1 阶段目标

本阶段完成基于神经网络的三档 AI 算法模块、状态建模，并补齐玩家成长系统与牌桌体验。实训任务要求扑克牌博弈 AI 至少提供两种可替换策略；项目最终采用 GoF 策略模式保留 AiStrategy 统一接口，在接口后接入端侧轻量 MLP 神经网络模型，使简单、普通、困难三档难度都能使用同一套模型能力。AI 策略、训练脚本和成长系统代码主要借助 Codex/GPT-5 生成后，再通过合法动作测试、随机局测试、benchmark 评估和人工对局验收进行修正。

7.2 AI 算法模块

AI 难度	策略配置	基本思路
简单	NeuralAiProfile.Easy (scoreNoise = 3.0)	共用 v3 MLP 对合法候选出牌评分，在分数上加入较大确定性扰动，降低压制性，适合新手练习
普通	NeuralAiProfile.Normal (scoreNoise = 0.8)	共用 v3 MLP，对模型判断做少量扰动，保持较稳定的控牌和出牌能力
困难	NeuralAiProfile.Hard (scoreNoise = 0.0)	使用完整神经网络分数，无扰动，优先选择模型认为最优的合法动作

AI 模块通过 AiStrategy 接口统一出牌决策入口。GameController 不直接依赖具体 AI 算法，只接收 chooseMove(state, playerId) 返回的 Move.Play 或 Move.Pass；ChudadiApp.strategy 根据 Difficulty 映射到 NeuralAiProfile.Easy、NeuralAiProfile.Normal、NeuralAiProfile.Hard。这种结构保留了 GoF 策略模式的可替换性，也把神经网络模型和牌局控制器隔离开。

当前生产模型为 mlp-selfplay-distilled-v3，是一个端侧轻量 MLP：输入层 23 个特征，隐藏层 16 个 tanh 单元，输出 1 个候选动作分数，共 401 个参数。模型权重以 Kotlin doubleArrayOf 形式内置在 NeuralMoveModel 中，并记录 WEIGHT_CHECKSUM，Android 端不需要 TensorFlow Lite、PyTorch Mobile 或网络请求，运行时只做普通数组点乘和 tanh 推理。

候选动作仍由 GameController.legalPlays() 枚举，神经网络不直接生成牌、不绕过规则。AI 在有一手出完的合法动作时优先结束对局；其他情况下，NeuralMoveFeatures.encode(...) 将每个合法候选动作编码成 23 维特征，再由 NeuralMoveModel.score(...) 评分，最后按模型分数、出牌张数、牌型强度和最高牌进行稳定排序。

23 维特征覆盖了当前出牌张数、牌型强度、主牌点数、最高牌点数和花色、出牌后剩余手牌、当前手牌数量、对手最少/最多手牌数、残局压力、过牌轮次、是否首轮、是否正在压牌、是否炸弹或五张牌、拆牌代价、低

牌/高牌占比、剩余牌组对子/三张/炸弹潜力、顺子潜力、南北规则炸弹增强开关、是否可以直接出完，以及是否正在回应强牌。这样模型输入既包含牌型信息，也包含局势压力和手牌结构。

训练脚本为 tools/train_neural_ai.py。离线训练使用固定 seed 20260610，包含 1800 条随机合成样本、32 局自对弈生成的 4427 条训练样本，以及 1686 条验证样本；训练轮数为 140。训练目标结合教师评分、出牌后局面价值和自对弈胜负回报，使模型学习“哪些合法动作更有利于尽快出完并减少结构损失”。当前训练清单记录在 docs/ai/neural_ai_training_manifest.json，验证集 MSE 为 0.105922，MAE 为 0.155211。

三档难度并不是三套彼此独立的旧规则策略，而是同一个神经网络模型的三种 profile。困难档 NeuralAiProfile.Hard 使用完整模型分数，scoreNoise = 0.0；普通档加入较小确定性扰动，scoreNoise = 0.8；简单档加入较大确定性扰动，scoreNoise = 3.0。扰动由当前局面和候选牌计算哈希得到，不依赖随机线程，因此同一状态下决策稳定，既能形成难度梯度，又不会破坏测试可复现性。

本项目在设计神经网络 AI 时参考了若干棋牌博弈 AI 资料。参考的重点不是直接复用框架代码，而是吸收其对“牌局状态、合法动作集合、Agent/策略接口、自对弈样本、不完全信息评估”的组织方式，并结合 Android 端运行条件自建轻量 MLP。

参考资料	资料信息	主要借鉴点	项目内落地方式
RLCard	面向纸牌游戏的强化学习工具包，支持 Blackjack、Leduc Hold'em、Texas Hold'em、UNO、Dou Dizhu、Mahjong 等环境。 GitHub: https://github.com/datamllab/rldcard 论文: https://arxiv.org/abs/1910.04376	“游戏环境 + 合法动作集合 + Agent 策略接口”的组织方式。	AI 不直接生成牌，先由 GameController.legalPlays() 取得合法候作，再由 AiStrategy.chooseMove() 选择动作。
OpenSpiel	Google DeepMind 开源的多智能体博弈研究框架，覆盖回合制/同步行动、完美/不完美信息、搜索和强化学习算法。 GitHub: https://github.com/google-deepmind/open_spiel	将游戏状态、当前玩家、合法动作、策略/Agent 解耦，便于多种算法共用同一游戏建模。	项目中 GameState、Move、AiStrategy 和 RuleSet 分离，控制器只调用接口，不绑定某个具体 AI 实现。
ISMCTS	Cowling、Powley、Whitehouse 关于 Information Set Monte Carlo Tree Search 的论文，研究隐藏信息和不确定性游戏中的搜索评估。 论文页面: https://pure.york.ac.uk/portal/en/publications/information-set-monte-carlo-tree-search/	不完全信息博弈中不能直接看见手手牌，需要基于可信息估候选动作。	特征设计中加入对手最少/最多手牌数、残局压力、是否正在压牌、是否回应强牌等可见局势信息，避免依赖不可见手牌。

参考资料	资料信息	主要借鉴点	项目内落地方式
DouZero	快手开源的斗地主 AI，论文题目为“DouZero: Mastering DouDizhu with Self-Play Deep Reinforcement Learning”。 GitHub: https://github.com/kwai/DouZero 论文: https://arxiv.org/abs/2106.06135	自对弈样本、出牌类游戏的大动作空间处理、候选动作编码与评估思想。	tools/train_neural_ai.py 使用 32 局自对弈生成 4427 条样本，并把候选动作 23 维特征后交给 MLP 评分。

因此，报告中“参考 RLCARD/OpenSpiel/DouZero/ISMCTS”表示项目在建模思想和训练样本组织方式上借鉴这些资料；实际训练和部署仍然采用本项目自写的 tools/train_neural_ai.py 训练脚本、Kotlin NeuralMoveModel 权重数组和 Android 端本地推理。

当前 benchmark 由 tools/benchmark_neural_ai.py 读取 Kotlin 生产权重后评估，避免只评估训练脚本内存模型。512 局对三名简单基线的评估结果为：简单档 168/512，胜率 0.328125；普通档 189/512，胜率 0.369141；困难档 204/512，胜率 0.398438；困档完局率 1.000000。教师策略一致性评估中，300 个状态精确一致率为 0.923333，Top 3 一致率为 0.996667。

为了防止报告、训练产物和 Kotlin 权重不一致，项目增加 tools/verify_neural_ai_artifacts.py。该脚本同时读取 NeuralAiStrategy.kt、docs/ai/neural_ai_training_manifest.json 和 docs/ai/neural_ai_benchmark_report.json，校验模型版本、权重 checksum、训练 seed、特征数、隐藏层规模、参数量、自对弈局数和 benchmark 阈值。当前 artifact_check=passed。

核心 Kotlin 接入片段如下：

```
enum class NeuralAiProfile(val title: String, val scoreNoise: Double) {
    Easy("简单", 3.0),
    Normal("普通", 0.80),
    Hard("困难", 0.0)
}

fun strategy(controller: GameController, level: Difficulty = difficulty): AiStrategy = when (level) {
    Difficulty.Easy -> NeuralAiStrategy(controller, NeuralAiProfile.Easy)
    Difficulty.Normal -> NeuralAiStrategy(controller, NeuralAiProfile.Normal)
    Difficulty.Hard -> NeuralAiStrategy(controller, NeuralAiProfile.Hard)
}
```

AI 决策安全边界：

1. AI 只在 GameController.legalPlays() 返回的合法候选动作中选择，输出动作后仍由 GameController 和 RuleSet 校验，防止非法牌进入牌局。
2. 蓝牙对局中 AI 仍只由房主端运行，客户端只接收房主快照，不各自运行模型，避免多设备决策分叉。
3. 神经网络输入使用公开牌局状态、当前玩家手牌和合法候选动作特征，不新增蓝牙协议字段，也不改变 RoomSeat.difficulty 的传输格式。
4. AI 思考期间 UI 显示思考状态，设置菜单打开时暂停 AI 行动，避免玩家打开菜单后 AI 继续出牌。

本阶段使用的 GRASP 原则和 GoF 设计模式如下：

GRASP/GoF 设计应用	阶段作用	代码或文档落点
GoF 策略模式 (Strategy)	三档 AI 均通过 AiStrategy.chooseMove 执行，具体难度由 NeuralAiProfile 配置	NeuralAiStrategy、NeuralAiProfile、AiStrategy
GRASP 多态 (Polymorphism)	简单、普通、困难 AI 通过同一 chooseMove 接口执行，调用处不需要关心神经网络内部结构	AiStrategy.chooseMove()、ChudadiApp.strategy()
GoF 状态模式 (State)	ReadyToStart、Dealing、Playing、Paused、Result 等阶段控制 AI 是否行动和界面是否可操作	GameStartPhase、Screen、状态图
GRASP 防变化 (Protected Variations)	AI 只对合法候选牌评分，最终仍由 GameController 和 RuleSet 校验，模型迭代不会破坏牌局规则	GameController.play()、RuleSet、NeuralMoveFeatures

7.3 状态建模

项目通过状态图描述从启动到结算的主要状态转换：

- 1. Splash：开屏页，玩家点击开始游戏后进入大厅。
- 2. Lobby：大厅，进入人机、蓝牙、规则、设置、教程或玩家中心。
- 3. ReadyToStart：人机对局预备牌桌，玩家确认开始。
- 4. Dealing：发牌动画阶段。
- 5. Playing：正式对局阶段，玩家和 AI 按回合出牌。
- 6. Paused：设置、确认或临时暂停状态。
- 7. Result：结算页，展示胜负、金币、经验、升级和再来一局。

状态建模帮助 UI 避免多个布尔值交叉控制，页面切换和牌局阶段都由明确状态驱动。

状态建模也用于修复 UI 与 AI 抢流程的问题。ReadyToStart 阶段不创建可行动状态，Dealing 阶段禁用出牌/不出/提示，Paused 阶段暂停倒计时和 AI 自动行动，Playing 阶段才允许 GameController 接收动作。这样解决了“打开设置菜单时 AI 继续出牌”“发牌动画未结束就开始倒计时”等体验问题。

状态图不仅描述页面，还描述关键业务保护：

状态	进入条件	禁用/允许内容
ReadyToStart	人机选择难度后进入牌桌预备页	不显示手牌，不启动倒计时，不触发 AI
Dealing	点击开始游戏后播放发牌动画	禁用出牌/不出/提示/重选，倒计时暂停
Playing	发牌动画完成后	允许当前玩家按规则出牌，AI 可以行动
Paused	设置菜单、弹窗确认、断线提示等情况	暂停倒计时和 AI 自动行动
ClientTakeover	客户端断线	房主 AI 托管断线座位并广播快照
HostDisconnected	房主断线或连接丢失	弹窗提示并退出当前房间，要求重新创建或重新加入

Result	玩家手牌清空	展示结算、奖励、成就和再来一局
--------	--------	-----------------

这些状态对应 GameStartPhase、Screen、BluetoothEntryMode、网络状态字符串和座位连接状态，共同保证 UI、倒计时、AI 和蓝牙同步不会互相抢流程。

7.4 玩家成长系统完善

玩家成长系统在本阶段完成，用于让大厅顶部头像、等级、金币、战绩入口从“静态展示”变成真实可用功能。该模块放在 profile/ 包中，使用 ProfileController 统一处理结算奖励、等级经验、金币消费、头像解锁、成就和战绩，使用 ProfileStore 通过 SharedPreferences 保存本地档案。

成长系统的主要功能如下：

功能	代码/数据	实现说明
玩家档案	PlayerProfile	保存昵称、头像、金币、经验、等级、称号、统计和成就
等级	ProfileController	每局根据名次和模式增加经验，经验达到阈值后升级
金币奖励	ProfileController.settleMatch	人机/蓝牙对局结束后根据名次发放金币
金币消费	renameWithCost、unlockAvatar、unlockCustomAvatar、resetStats	改名、头像解锁、自定义头像、重置战绩都需要金币并二次确认
头像商店	BuiltInAvatar	内置多个自绘头像，按普通、稀有、传说设置不同价格和视觉样式
自定义头像	相册选择 + 私有目录复制	未解锁时显示明显提示，解锁后可选择本地图片作为头像
战绩	MatchRecord、ProfileStats	记录最近 50 局，总局数、胜场、蓝牙局、困难胜场、连胜等统计
成就	Achievement	首胜、连胜、困难胜利、蓝牙对局等条件触发，只奖励一次
每日奖励	ProfileController.claimDailyReward	同一天不可重复领取，跨天可再次领取

玩家成长结算流程：

1. 对局结束后，结算页根据名次、模式、难度、规则和剩余牌数生成 MatchSettlement。
 2. ProfileController.settleMatch(profile, settlement) 计算金币、经验、成就和历史记录。
 3. 控制器返回 ProfileChange，其中包含更新后的 PlayerProfile、本局 MatchRecord 和提示信息。
 4. ProfileStore.saveProfile() 写入 SharedPreferences。
 5. 大厅、玩家中心和结算页读取同一份档案，显示最新金币、等级、头像和战绩。
- 金币消费采用确认弹窗保护：玩家点击改名、解锁头像、解锁自定义头像或重置战绩时，先显示消费说明和金币数量；确认后才调用 ProfileController，金币不足时不修改档案并弹窗提示失败原因。这样可以避免误扣金币，也方便答辩时说明本地数据的完整闭环。

头像商店和自定义头像也采用“先确认、后扣费、再写入”的流程。未解锁头像会显示明显的价格和锁定状态，自定义头像功能需要一次性金币解锁；解锁后选择相册图片会复制到 App 私有目录，图片失效时回退到内置头像。这些逻辑使大厅顶部头像、昵称、等级和金币成为真实可交互系统，而不是静态装饰。

成长系统的代码亮点是 ProfileController 返回 ProfileChange，而不是让页面直接修改金币、经验或战绩。ProfileChange 同时携带更新后的 PlayerProfile、提示消息、升级和成就信息，UI 只负责展示弹窗和刷新状态。ProfileStore 负责 SharedPreferences 序列化，旧档案字段缺失时会补默认头像、金币和解锁状态，减少版本迭代后本地数据读取失败的风险。

本阶段为成长系统使用的 GRASP 原则和 GoF 设计模式如下：

GRASP/GoF 设计应用	代码落点	说明
GRASP 信息家 (Information Expert)	PlayerProfile、ProfileStats、MatchRecord	玩家档案、统计和历史记录由掌握相关数据的对象承载
GRASP 控制器 (Controller)	ProfileController	统一处理结算、改名、头像解锁、战绩重置和每日奖励等用户操作
GRASP 纯虚构 (Pure Fabrication)	ProfileStore	SharedPreferences 保存读取不属于 UI 或领域对象职责，因此拆为独立持久化对象
GoF 外模式 (Facade) / GRASP 防变化 (Protected Variations)	ProfileController、ProfileChange、金币不足判断	对 UI 提供统一入口；消费失败时不改状态、不扣金币、不写入存储

7.5 UI 与交互完善

本阶段继续完善 Compose 页面和动画体验：

页面	功能
开屏页	展示扑克牌背景和开始游戏入口
大厅	显示玩家头像、昵称、等级、金币和主要功能入口
人机难度页	选择简单、普通、困难 AI
规则	切换北方和南方规则，并保存到本机
蓝牙房间	创建房间、加入对局、四座位、人机补位和准备状态
牌桌页	发牌、手牌选择、出牌、不出、提示、倒计时和 AI 思考
结算页	显示胜负、金币经验变化、成就和再来一局
玩家中心	昵称、头像、金币、等级、战绩、成就、每日奖励和头像商店

UI 技术实现细节：

1. 页面切换由 Screen 状态控制，不引入 Navigation 依赖，降低 Gradle 依赖风险。
2. AnimatedScreenHost 对页面切换做淡入、滑入和轻微缩放，避免硬切换。
3. 大厅使用漂浮扑克牌、主视觉光效、面板阴影和金色按钮，表现棋牌大厅氛围。
4. 人机开局采用“先入桌、再开始”的流程，玩家点击 开始游戏 后才发牌，和正式棋牌游戏流程更接近。
5. 发牌动画使用单个全局进度驱动中心牌堆向四个座位飞牌，避免 52 张牌各自启动独立动画导致卡顿。
6. 手牌选中时上浮、加阴影、轻微放大；出牌成功后播放飞向中央牌桌的动画。
7. 无可压牌时显示“手上没有可以大过人家的牌”，禁用出牌和提示，只保留不出。
8. 设置菜单打开时暂停倒计时和 AI 行动，菜单项统一为继续游戏、重新开始、返回大厅和退出确认。
9. 结算页展示金币、经验、升级、成就和再来一局；蓝牙对局再来一局回到原房间，不要求重新加入。

界面卡顿优化是本阶段重点之一。早期版本中，大厅主视觉上移动画、漂浮扑克牌、牌桌光效、手牌发牌、卡牌阴影和结算粒子可能同时运行。Jetpack Compose 是状态驱动界面，状态变化会触发重组和重新绘制；如果每张牌、每个光效都各自维护动画状态，手机每一帧都要处理大量状态写入、布局测量和绘制任务，就会出现大厅动画一顿一顿、发牌结束后手牌跳动、置牌区突然变大的问题。优化时没有简单追求更高帧率，而是先减少每帧需要变化的状态数量。

具体实现上，发牌动画使用单个全局 Animatable 进度驱动 52 张牌的位置计算，不为每张牌单独创建协程和动画对象；所有牌的飞行位置都由同一个进度值、发牌序号和目标座位推导出来，因此每帧只更新一个核心动画状态。大厅漂浮牌和光效改为固定数量 Canvas/graphicsLayer 绘制，尽量只改变位移、缩放和透明度，不频繁改变布局尺寸。手牌列表则使用稳定 key：每张牌用花色和点数组成唯一身份，例如 Spade-3、Heart-A。这样玩家选牌、提示、排序或出牌后，Compose 能识别“这还是同一张牌，只是位置或状态变了”，不会把整组手牌当成新组件重新创建，从而减少选中上浮、阴影、飞牌动画的抖动和重复播放。

同时，动画牌和真实手牌统一使用同一套卡牌尺寸、间距和牌桌区域约束。发牌动画结束后，牌背飞到的位置与底部真实手牌展开位置在视觉上连续，中央置牌区也保持固定高度，不再因为上一手牌数量变化而反复扩大或压缩。复杂粒子只在结算弹层播放一次，普通大厅和牌桌保留轻量循环动画，优先保证低端手机和模拟器上的稳定流畅。

卡顿修复中也使用了 AI 辅助分析。开发者把“回到大厅时主视觉动画一顿一顿”“发牌动画和真实手牌大小割裂”“置牌区莫名变大”等现象描述给 Codex/GPT-5，模型辅助排查动画状态、布局尺寸和重组触发点。最终采用统一卡牌尺寸、稳定牌桌区域约束、减少无限动画数量、把复杂粒子限制在结算场景等方案，优先保证低端手机和模拟器上的可用流畅度。

音乐和震动同样做了生命周期控制。CardRoomMusicPlayer 统一管理背景音乐，避免页面切换时重复创建 MediaPlayer；设置页的开关只改变播放状态，不影响 GameController。出牌音效和震动只在成功出牌时触发，非法出牌或无可压牌时不播放反馈，避免玩家误解操作结果。

音乐与反馈系统：

1. 背景音乐使用欢乐斗地主背景音乐.ogg，并按 0:00 到 1:33 截取循环播放。
2. 主界面和对局中都能播放背景音乐，设置页可以开关。
3. 出牌时触发音效，震动开关打开时触发系统震动反馈。
4. CardRoomMusicPlayer 负责背景音乐生命周期，避免页面切换后重复创建播放器。
5. 音效和震动属于用户反馈，不参与牌局逻辑，因此与 GameController 分离。

7.6 实现成果

本阶段完成三档神经网络 AI、玩家成长系统、状态图细化、牌桌交互和用户体验增强。测试中覆盖神经网络模型元数据、特征维度、AI 合法动作、三档 profile 随机多局完成、玩家成长和动画 key 稳定性，减少 AI 出非法牌、成长数据丢失或 UI 动画重复播放的风险。

阶段成果展示：

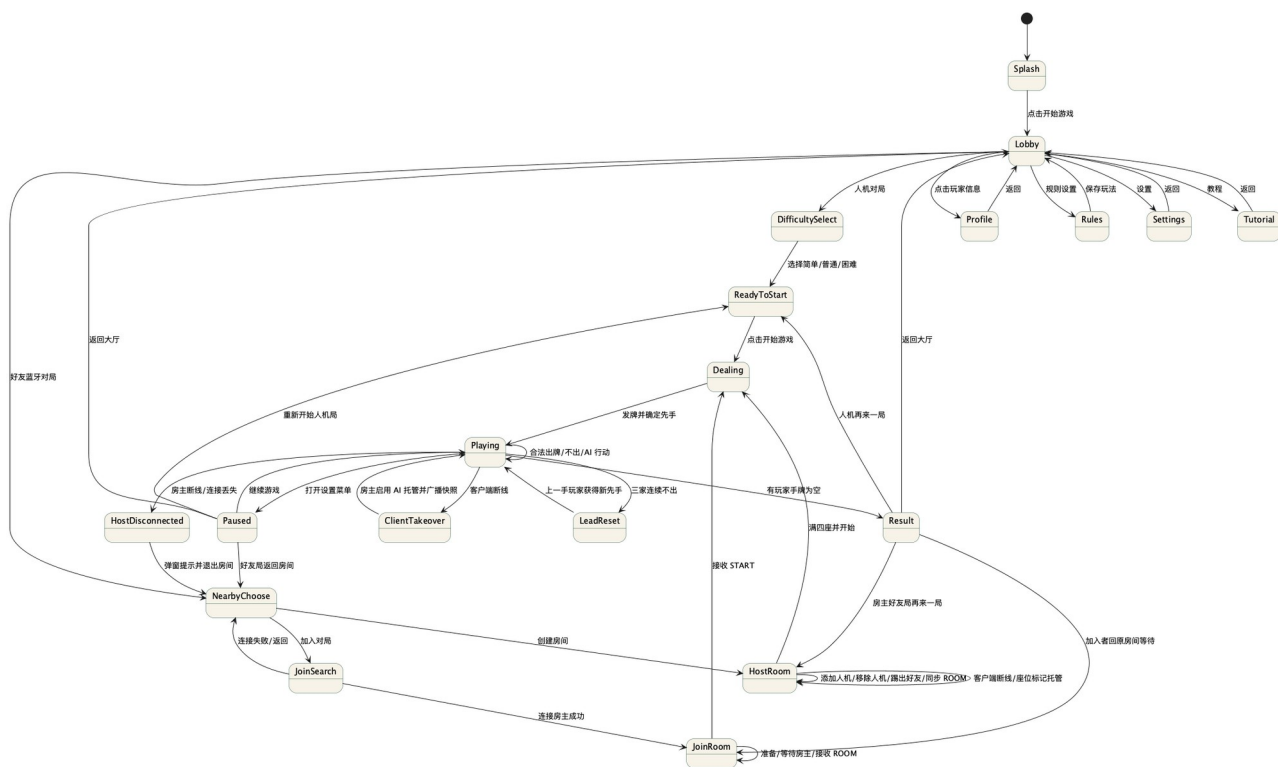
成果类型	展示内容	证据材料
Code	NeuralAiStrategy 统一简单、普通、困难三档 AI，并内置 NeuralMoveModel v3 权重	app/src/main/java/com/example/uml_chudadi/controller/NeuralAiStrategy.kt
Code	ProfileController 和 ProfileStore 管理金币、经、头像、和成就	app/src/main/java/com/example/uml_chudadi/profile/
Code	Screen 和 GameStartPhase 管理页面切换、人机预备、发牌、游戏中、结算等状态	app/src/main/java/com/example/uml_chudadi/view/ChudadiApp.kt
Image	状态图展示开屏、大厅、房间、发牌、游戏中、暂停、断线提示和结算状态	docs/uml/state_game.png
成果	AI 合法动作、随机多局完成、玩家成长、动画 key 稳定性测试	GameControllerTest、ProfileControllerTest、TableAnimationKeysTest
UI 成果	大厅、人机难度、规则、蓝牙房间、牌桌、结算、玩家中心等页面可切换	ChudadiApp.kt

神经网络 AI 接入代码片段如下：

```
enum class NeuralAiProfile(val title: String, val scoreNoise: Double) {
    Easy("简单", 3.0),
    Normal("普通", 0.80),
    Hard("困难", 0.0)
}

fun strategy(controller: GameController, level: Difficulty = difficulty): AiStrategy = when (level) {
    Difficulty.Easy -> NeuralAiStrategy(controller, NeuralAiProfile.Easy)
    Difficulty.Normal -> NeuralAiStrategy(controller, NeuralAiProfile.Normal)
    Difficulty.Hard -> NeuralAiStrategy(controller, NeuralAiProfile.Hard)
}
```

阶段图片成果如下：



问题	解决方法
AI 难度差异不明显	共用同一神经网络模型， 但通过 Easy/Normal/Hard 三种 NeuralAiProfile 设置不同 scoreNoise， 并用 benchmark 验证简单、普通、困难胜率梯度
模型效果需要可量化证明	增加训练清单、benchmark 报告和 artifact 校验脚本， 记录胜率、完局率、教师策略一致率和权重 checksum
UI 状态复杂	使用 Screen、BluetoothEntryMode、GameStartPhase 管理页面和开局状态
成长数据容易分散在页面中	使用 ProfileController 和 ProfileStore 集中处理结算、消费和持久化
动画容易重复触发	使用 TableAnimationKeysTest 出牌动画 key 稳定

8.2 系统整合

交付阶段首先进行系统整合，把前面三个细化迭代中分别完成的规则、AI、蓝牙、UI、玩家成长、音乐反馈和本地存储统一接入最终 App。整合顺序不是直接打包发布，而是先确认各模块之间的数据流、状态流和页面入口能够连成完整闭环。

整合阶段继续使用 Codex/GPT-5 辅助检查跨模块数据流。开发者将入口页面、规则保存、人机开局、蓝牙房间、结算奖励和 Release 配置串成完整路径后，把发现的问题和代码片段交给模型分析，例如规则保存后是否影响蓝牙 START 消息、结算后再来一局是否应回原房间、Release 与 Debug 的运行配置是否混淆。AI 给出的修改建议最终都需要通过本地构建和真机运行确认。

最终项目整合了以下功能：

- 1. 开屏页、大厅、规则、设置、教程、玩家中心、人机对局、蓝牙房间、牌桌和结算页面。
- 2. 南北规则切换与本地保存。
- 3. 简单、普通、困难三档 AI。
- 4. 多人蓝牙房间、四座位、人机补位、房主权威同步、客户端断线 AI 托管，以及房主断线弹窗退出。
- 5. 玩家成长系统，包括等级、经验、金币、称号、战绩、成就、每日奖励、头像商店、自定义头像和改名消费。
- 6. 背景音乐、出牌音效、震动、开屏动画、发牌动画、出牌飞牌和结算动画。
- 7. Release APK 和 AAB 包。

8.3 代码测试与验收

系统整合后第二步是测试代码，而不是立刻发布。测试分为本地单元测试、蓝牙模拟测试、构建测试、设备冒烟测试和手动验收五类，目标是尽量在发布 Release 前排除规则、状态、协议、成长数据和安装运行问题。

测试用例设计也借助 AI 扩展场景。除了常规牌型识别测试外，Codex/GPT-5 帮助补充了固定 seed 随机多局、非法出牌、三家过牌重置、无可压牌提示、重复 MOVE_REQUEST 过滤、客户端断线托管、金币不足消费失败、头像重复解锁不重复扣费、动画 key 稳定性等边界用例。开发者根据实际源码保留可运行测试，删除无法在本地环境稳定复现的设想。

修 bug 时的流程是“复现现象—收集日志—让 AI 辅助分析—修改代码—重新测试”。例如蓝牙问题会同时检查传输层事件、房间座位、UI 弹窗和消息协议；动画问题会同时检查 Compose 状态、key、尺寸约束和无限动画数量；Release 问题会检查签名、keystore 忽略规则和 runRelease 配置。这个流程比只看界面现象更可靠。

测试文件	覆盖内容	结论
------	------	----

HandClassifierTest	单张、对子、三张、顺子、同花、葫芦、四带一、同花顺、南北规则差异	覆盖核心牌型和规则差异
GameControllerTest	发牌、首出、过牌重置、神经网络模型元数据、AI 合法动作、三档 profile 固定 seed 随机多局完成	覆盖牌局主流程，防止死循环、非法状态和模型权重/特征维度不一致
PlayerActionPolicyTest	无可压牌提示、按钮可用性、等待确认、发牌阶段锁定	覆盖玩家操作是否可点击、是否应提示
GameMessageCodecTest	ROOM、START、MOVE、SNAPSHOT、DISCONNECT_NOTICE 等消息往返	覆盖联机协议编码/解码和中文昵称场景
RoomSeatTest	四座位、添加人机、准备、客户端断线托管、再开局重置	覆盖蓝牙房间座位逻辑
NetworkMoveGuardTest	当前远端玩家校验、重复/非法请求过滤	覆盖房主权威保护，避免重复点击污染牌局
SimulatedBluetoothHubTest	三人加入、房满拒绝、房主广播、移动请求和客户端断线托管	覆盖多人蓝牙核心逻辑的可重复模拟
ProfileControllerTest	经验升级、金币结算、头像解锁、改名消费、战绩重置、每日奖励	覆盖玩家成长系统和金币消费规则
TableAnimationKeysTest	出牌动画 key 和落桌动画 key 稳定性	覆盖动画重复触发风险
MusicAssetTest	背景音乐资源存在和循环配置	覆盖音乐资源和 0:00-1:33 循环要求
ChudadiDeviceSmokeTest	真机/模拟器启动和基础 UI 冒烟	覆盖设备启动验收

测试执行命令按以下顺序进行：

```
./gradlew --stop
./gradlew test --no-daemon
./gradlew :app:assembleDebug --no-daemon
./gradlew :app:assembleRelease --no-daemon
./gradlew :app:bundleRelease --no-daemon
```

测试方法说明：

1. 规则类测试放在本地单元测试中，运行快、可重复，不依赖手机。
2. 蓝牙核心逻辑通过 SimulatedBluetoothHubTest 模拟，解决多台真机不足时无法复现并发加入、客户端断线托管等问题。
3. 构建测试分 Debug 和 Release 两条线，Debug 证明开发环境能编译运行，Release 证明发布包能生成。
4. 设备冒烟测试放在 androidTest，用于确认 App 可启动并显示主界面。
5. 手动验收用于覆盖真机权限弹窗、蓝牙系统行为、音乐播放、动画观感和玩家实际操作路径。

6. 文档和 UML 在测试后同步核对，避免答辩时“图、文、代码、运行效果”不一致。

场景	预期结果
开屏进入大厅	点击开始游戏后进入大厅，UI 无明显错位
人机对局	简单、普通、困难均可开局并完成一局
南北规则	保存规则后影响后续人机和蓝牙对局
蓝牙房间	房主创建房间、加入者连接、四座位准备、人机补位、客户端断线托管、房主断线弹窗退出和开局
出牌流程	玩家可选牌、出牌、不出、提示，非法牌显示提示
无可压牌	显示“手上没有可以大过人家的牌”，只允许不出
结算	显示胜负、金币、经验、升级和再来一局
玩家中心	改名、头像、每日奖励、战绩和成就逻辑正常
音乐与反馈	背景音乐循环、出牌音效、震动开关生效

测试完成后，才进入 Release 发布和托管流程。

8.4 Release 发布与托管

测试通过后第三步才是发布 Release。Release 发布不是简单复制 APK，而是要确认签名、包路径、敏感文件、GitHub 托管和一键运行配置。

项目	当前交付
包名	com.example.uml_chudadi
Release APK	release/chudadi-v1.3-release.apk
Release AAB	release/chudadi-v1.3-release.aab
GitHub 远程	https://github.com/Mojieeeee/uml_chudadi.git
常用测试命令	<code>./gradlew test --no-daemon</code>
Debug 构建	<code>./gradlew :app:assembleDebug --no-daemon</code>
Release 构建	<code>./gradlew :app:assembleRelease --no-daemon</code>
Release Bundle	<code>./gradlew :app:bundleRelease --no-daemon</code>

Release 发布流程如下：

1. 确认 `./gradlew test --no-daemon`、Debug 构建和 Release 构建已经通过。
2. 执行 `./gradlew :app:assembleRelease --no-daemon` 生成 Release APK。

- 3. 执行 `./gradlew :app:bundleRelease --no-daemon` 生成 AAB。
- 4. 使用 `apksigner verify --verbose --print-certs app/build/outputs/apk/release/app-release.apk` 检查签名。
- 5. 将 APK/AAB 复制或归档到 `release/` 目录，作为测试分发包。
- 6. 使用 `.idea/runConfigurations/runRelease.xml` 在 Android Studio 中一键安装 Release 包到手机。
- 7. 使用 GitHub HTTPS 上传项目源码，确保 `.jks` 私钥和 `keystore.properties` 不进入仓库。

Release 私钥、`keystore.properties`、`local.properties`、`.gradle`、`build/` 等敏感或构建目录不上传 GitHub。

8.5 问题与解决

问题	解决方法
真机蓝牙受品牌、权限、配对状态影响	自动化模拟测试覆盖协议和房间逻辑，答辩前再做真机复测
蓝牙断线场景难以稳定人工复现	使用模拟传输覆盖客户端断线托管；房主断线采用真机/手动验收确认弹窗退出
Debug 和 Release 签名不同	使用 <code>runRelease</code> 自动卸载旧包并安装 Release
敏感文件不能上传	将签名私钥和本地配置排除在 GitHub 之外

9. 实验结果展示与录屏说明

9.1 功能展示内容

最终答辩录屏采用“完整流程 + 分段展示”的方式。完整流程用于展示项目可连贯运行，分段录屏用于突出人机、蓝牙、规则、玩家成长、测试构建和发布包等关键验收内容。

录屏名称	推荐文件名	展示内容	展示重点	验证结论
完整流程屏	<code>demo_full_flow.mp4</code>	开屏、进入大厅、规则选择、人机对局、出牌、结算、玩家中心	UI、规则、AI、玩家成长	证明 App 主流程完整可运行
人机与 AI 录屏	<code>demo_ai_match.mp4</code>	简单/普通/困难难度选择，神经网络 AI 自动出牌	三档神经网络 AI、策略模式、难度梯度	证明 AI 模块可替换并参与对局
牙房间屏	<code>demo_bluetooth_room.mp4</code>	创建房间、加入对局、四座位、人机补位、准备开局、客户端断线托管和房主断线提示	多人蓝牙连接	证明联机不是一对一匹配，并考虑断线处理
南北屏	<code>demo_rules.mp4</code>	切换北方/南方规则并进入对局验证首出差异	南北规则	证明规则可自由选择并保存
玩家中心屏	<code>demo_profile.mp4</code>	金币、等级、战绩、成就、每日奖励、头像和改名确认	UI、玩家成长、持久化	证明扩展体验和本地数据有效
构建屏	<code>demo_test_build.mp4</code>	Gradle test、Debug 构建、	测试与交付	证明交付质量和发布包有效

		Release 构建或签名检查		
--	--	-----------------	--	--

项目材料：仓库已包含 UML 图片和 Release 包。App 运行截图、真机蓝牙截图和演示录屏在最终提交前归档，并在 PPT 中引用对应文件或截图。

9.2 截图材料

截图内容	目的
开屏页和大厅	展示发布版 UI 风格
规则选择页	展示南北规则自由选择
人机难度页	展示三档 AI
牌桌页	展示发牌、手牌、出牌、不出和提示
蓝牙房间页	展示多人房间和四座位
结算页	展示对局结果和玩家成长
玩家中心	展示金币、等级、战绩、成就和头像
Gradle 果	展示通据
Release 包目录	展示最终交付物

10. 总结与收获

通过本次软件综合开发实训，小组完整经历了需求分析、UML 建模、面向对象设计、编码实现、测试验收和发布交付的过程。

在 UML 和软件工程方面，本小组理解并实践了 UP 迭代开发中“从用例到领域模型，再到设计模型和实现”的过程。通过用例图、类图、顺序图、状态图、活动图、组件图和部署图，项目从需求到代码具备可解释、可追踪的模型支撑。

在 Android 开发方面，本小组熟悉了 Android Studio、Gradle、Kotlin 和 Jetpack Compose。项目采用单 Activity + Compose 基于状态的页面切换的方式，实现了多个页面、动画、音效和本地数据保存。

在面向对象设计方面，项目通过 MVC 分层架构，以及 GRASP 信息专家、控制器、高内聚、低耦合、间接性、防变化、纯虚构和 GoF 策略、适配器、命令、观察者、状态、外观等模式降低复杂度。规则、AI、蓝牙和玩家成长模块都可以被独立测试和维护。

在蓝牙和 AI 模块方面，项目实践表明多人联机游戏不仅需要连接成功，还需要解决状态同步、重复请求、当前玩家校验和断线恢复等问题。AI 模块也不仅是随机出牌，而是需要通过策略模式、特征建模和神经网络评分形成不同难度和可替换算法。

在测试和交付方面，项目通过单元测试、模拟蓝牙测试、构建测试、Release 签名包和手动验收表提高可信度。测试工作帮助小组发现并规避了规则、消息、座位、动画和玩家成长中的潜在问题。

在 AI 工具使用方面，本项目把 Codex/GPT-5 作为开发辅助引擎使用：需求阶段用于拆解功能边界和迭代路线，设计阶段用于比较 MVC 分层、蓝牙 C/S 房间、规则策略和本地成长系统方案，建模阶段用于生成 PlantUML 草图并辅助迁移到 ProcessOn，编码阶段用于生成 Kotlin/Compose、蓝牙协议、AI 策略、ProfileController 和测试初稿，调试阶段用于根据 Build Output、Logcat 和复现场景分析 bug，测试阶段

用于补充单元测试、模拟蓝牙测试和边界用例。AI 工具显著提高了实现速度，但最终交付仍以 Android Studio 编译、Gradle 测试、真机/模拟器运行和图文代码一致性为准。

11. 附录：主要材料清单

类型	文件或目录	说明
项目完整说明	docs/complete_project_documentation.md	覆盖代码结构、UML、GRASP/GoF 设计应用、测试和发布说明
需求分析	docs/requirements.md	项目目标、功能需求、非功能需求和要求点映射
设计说明	docs/design.md	MVC 分层、核心职责、GRASP/GoF 设计应用、数据流和错误处理
测试报告	docs/test_report.md	自动化测试范围、验证命令、手动验收场景和风险
AI 使用说明	docs/ai_usage.md	AI 工具使用阶段、效果评价和开发者确认与调整内容
SCRUM 记录	docs/scrum.md	迭代目标和风险处理
最终检查表	docs/final_checklist.md	要求点、交付物和提交前检查项
UML 图	docs/uml/	7 类 UML 图的 PlantUML 源文件和 PNG 导出图
Release 包	release/	chudadi-v1.3-release.apk 和 chudadi-v1.3-release.aab
Android 代	app/src/main/java/com/example/ uml_chudadi/	Kotlin/Compose 主体代
代	app/src/test/、app/src/androidTest/	单元测试、模拟测试和设备冒烟测试